Final

Graded

Student

TIYA CHOKHANI

Total Points

88 / 100 pts

Question 1

(no title) **24** / 28 pts

1.1 (no title) **7** / 7 pts

✔ **– 0 pts** Correct

– 3 pts Major issue: divided difference definitions missing/incorrect (one or both).

– 2 pts Major issue: approximation is wrong/unclear (confuses derivative order, missing factor 1/2 idea for 2nd DD).

– 2 pts Exactness/theorem/conditions missing or substantially incorrect (no meaningful statement about when/where equality holds).

– 1 pt Minor issue: mostly correct but missing a key condition (e.g., differentiability assumptions) or has a small notational confusion.

1.2 (no title) **8** / 8 pts

✔ **– 0 pts** Correct

– 2 pts Linear interpolant $p(x)$ not correctly written in Newton/divided-difference form.

– 4 pts Secant iteration not obtained (or formula fundamentally wrong / not solvable to the secant update).

– 2 pts Newton connection missing/incorrect (fails to mention derivative replaced by divided difference / difference quotient).

– 1 pt Minor algebra/sign errors.

1.3 (no title) **5** / 5 pts

✔ **– 0 pts** Correct

– 2 pts First divided difference computed using wrong formula/incorrect conceptual setup.

– 3 pts Next iterate x_2 not computed from secant correctly (wrong update formula / wrong substitution).

– 1 pt Minor algebra/sign errors.

1.4 **(no title)** **4 / 8 pts**

- **− 0 pts** Correct
- ✔ **− 3 pts** Does not produce a product-type error relation involving e_n, e_{n-1} and a ratio of divided differences (or equivalent).
- **− 2 pts** Error relation is mostly incorrect (contains ratio of differences or error products)
- **− 1 pt** Error relation is partially incorrect (contains product and differences)
- ✔ **− 1 pt** Does not correctly replace the divided differences using the “exact” characterization from (a) (e.g., MVT forms) / confuses which DD corresponds to which derivative order.
- **− 1 pt** Qualitative convergence comparison missing/incorrect (secant vs bisection vs Newton).
- **− 1 pt** Mostly correct but has only a superficial/unclear justification.
- **− 8 pts** Blank

Question 2

(no title) **20** / 22 pts

2.1 (no title) **5** / 5 pts

✔ **– 0 pts** Correct

– 4 pts Wrong stencil / wrong derivative approximation (not centered 3-point derivative at x_0

– 3 pts Interpolation setup incomplete/incorrect (but intent is correct).

– 2 pts Final coefficients are incorrect (method otherwise is ok)

– 1 pt Minor algebra error in deriving coefficients (clearly close to correct generally in form 1/h).

2.2 (no title) **6** / 7 pts

– 0 pts Correct

– 4 pts Follow through

– 2 pts Correct general idea but missing/incorrect constant/derivative identification (e.g., wrong controlling derivative).

✔ **– 1 pt** Order-of-accuracy statement missing/incorrect (even if derivation mostly right).

– 1 pt Minor algebra error but correct conclusion.

2.3 (no title) **7** / 8 pts

– 0 pts Correct

– 3 pts Uses an incorrect differentiation formula for $f'(0)$.

– 2 pts One or both approximations for $h = 0.1$ or $h = 0.5$ incorrectly substituted

– 2 pts Error expressions incorrect

✔ **– 1 pt** Order check/comment missing/incorrect

– 2 pts Follow through

2.4 (no title) **2** / 2 pts

✔ **– 0 pts** Correct

– 2 pts Does not identify subtraction/cancellation and roundoff amplification as the key issue for very small h

– 1 pt Does not mention the truncation–roundoff tradeof

Question 3

(no title) **25** / 25 pts

3.1 **(no title)** **8** / 8 pts

- ✔ **– 0 pts** Correct
- **– 4 pts** $T(1)$ incorrect (wrong trapezoid rule setup).
- **– 4 pts** $T(1/2)$ incorrect.
- **– 2 pts** Minor structural error
- **– 1 pt** algebra error
- **– 8 pts** blank

3.2 **(no title)** **9** / 9 pts

- ✔ **– 0 pts** Correct
- **– 3 pts** Incorrect expansion for $I - T(h/2)$ from the given expansion (wrong powers or coefficients).
- **– 4 pts** Extrapolation coefficients A, B not correctly determined.
- **– 2 pts** Does not convincingly conclude improved order
- **– 1 pt** Minor algebra error
- **– 1 pt** Partial gets A=-4B
- **– 9 pts** Blank

3.3 **(no title)** **4** / 4 pts

- ✔ **– 0 pts** Correct
- **– 2 pts** $R(1)$ not computed correctly.
- **– 1 pt** Algebra errors/follow through
- **– 2 pts** Comparison/explanation missing
- **– 1 pt** Comparison wrong/unclear
- **– 4 pts** blank

3.4 **(no title)** **4** / 4 pts

- ✔ **– 0 pts** Correct
- **– 2 pts** Romberg tableau structure/placement incorrect (doesn't identify where the first extrapolated value goes).
- **– 2 pts** Hermite-tableau analogy missing/incorrect (no clear "moving right reuses previous entries and raises order" idea).
- **– 1 pt** Vague but directionally correct explanation.

Question 4

(no title) **19** / 25 pts

4.1 **(no title)** **3** / 5 pts

- **− 0 pts** Correct
- ✔ **− 2 pts** LU condition incorrect
- **− 1 pt** LU partial (sufficient conditions including diagonally dominant or positive definite)
- **− 2 pts** Cholesky condition incorrect
- **− 1 pt** Forms of factors unclear

4.2 **(no title)** **1** / 5 pts

- **− 0 pts** Correct
- **− 2 pts** LU condition for the specific A not justified.
- **− 1 pt** LU condition follow through
- ✔ **− 3 pts** Sum-of-squares / SPD argument incorrect or incomplete
- **− 2 pts** Matrix algebra stopped/errors
- **− 1 pt** SPD incomplete to sum of squares.
- ✔ **− 1 pt** Algebra errors in sum-of-squares but positivity conclusion is still supported.

4.3 **(no title)** **7** / 7 pts

- ✔ **− 0 pts** Correct
- **− 3 pts** L incorrect in a major way (wrong multipliers / not unit lower triangular / inconsistent with elimination).
- **− 3 pts** U incorrect in a major way (wrong pivots/superdiagonal / inconsistent with elimination).
- **− 1 pt** Minor arithmetic error
- **− 7 pts** Blank

4.4 **(no title)** **8** / 8 pts

- ✔ **− 0 pts** Correct
- **− 3 pts** Diagonal incorrect
- **− 2 pts** Incorrect LDL form
- **− 2 pts** Incorrect "square root" of D.
- **− 1 pt** Final Cholesky unclear
- **− 8 pts** Blank

Exam policies. You may use one double-sided 8.5″ × 11″ **sheet of notes. No electronic devices or calculators are allowed.**

All of your solutions should be easily identifiable and supporting work must be shown. Please clearly label what work you want to be considered for grading. Ambiguous or illegible answers will not be counted as correct.

The duration of the exam is 180 minutes. If you finish early, feel free to submit your exam and leave the room, but not in the last 5 minutes.

Name: Tiya Chokhani

Student ID Number: 305933966

Signature:

I certify, on my honor, that I have neither given nor received any help, or used any non-permitted resources while completing this evaluation.

1. (25 points) **Divided Differences:** The goal of this question is to revisit the secant method using the tools and techniques of divided differences.

   (a) (7 points) For a given function $f(x)$ defined on three distinct points x_0, x_1, x_2, consider the divided differences $f[x_0, x_1]$ and $f[x_0, x_1, x_2]$. These two quantities approximate properties of $f(x)$ of different *orders*.

   - Define each of the divided differences in terms of $f(x_i)$.
   - Clearly state what property of $f(x)$ each term approximates (for example, $f[x_0, x_1] \approx ?$) and of what order.
   - Determine if there is any value for which the approximation is *exact* (for example $f[x_0, x_1] = ?$) and state any necessary theorems associated and/or conditions on that value.

$f[x_0,x_1] = \frac{f[x_1]-f[x_0]}{x_1-x_0}$ this is a first order divided diff. The error is $f(x)-P_n(x) = \frac{f^{n+1}(\xi)}{(n+1)!}\Psi_n$ so for first order diff $n=1$ the error has $f''(\xi)$ so for $f(x)$ that are linear & $f''(x)=0$ the approximation is exact. For second order ($f[x_0,x_1,x_2]$) $f'''(x)$ must $=0$ so up to quadratic $f(x)$ is exact

$f[x_1,x_2] = \frac{f[x_2]-f[x_1]}{x_2-x_1}$ ~~this is a first order divided diff~~

$f[x_0,x_1,x_2] = \frac{f[x_1,x_2]-f[x_0,x_1]}{x_2-x_0}$ this is a second order divided diff.

$P_n(x) = a_0 + a_1(x-x_0) + a_2(x-x_1)(x-x_0)$
where $a_1 = f[x_0,x_1]$ & $a_2 = f[x_0,x_1,x_2]$

   (b) (8 points) Let $p(x)$ be the linear interpolating polynomial for f through the points $(x_{n-1}, f(x_{n-1}))$ and $(x_n, f(x_n))$, where $x_{n-1} \neq x_n$ and $f(x_{n-1}) \neq f(x_n)$.

   - Write $p(x)$ using the divided difference $f[x_{n-1}, x_n]$ (*that is, in Newton form*).
   - Set $p(x_{n+1}) = 0$ and solve for x_{n+1}. Show that this yields the secant iteration written in terms of the divided difference.
   - Briefly explain the relationship between the Secant method in this form and Newton's method.

$p(x) = f[x_{n-1}] + f[x_{n-1}, x_n](x - x_{n-1})$

$p(x_{n+1}) = f[x_{n-1}] + f[x_{n-1}, x_n](x_{n+1} - x_{n-1}) = 0$

$p(x_{n+1}) = f(x_{n-1}) + \frac{f(x_n)-f(x_{n-1})}{x_n - x_{n-1}}(x_{n+1}-x_{n-1}) = 0$ | $p(x_{n+1})=0$ so x_{n+1} is the root of $f(x)$

$(x_{n+1} - x_{n-1}) = [\cancel{p(x_{n+1})}^{0} - f(x_{n-1})]\frac{x_n - x_{n-1}}{f(x_n)-f(x_{n-1})}$

$x_{n+1} = x_{n-1} - f(x_{n-1})\frac{x_n - x_{n-1}}{f(x_n)-f(x_{n-1})}$

Substituting $f'(x_{n-1}) = \frac{f(x_{n-1})-f(x_{n-2})}{x_{n-1}-x_{n-2}}$ which is like a $f[x_{n-1}, x_{n-2}]$ divided diff in to newtons gives us secants method. So that no derivatives are needed & it's an approx for it. This is cheaper to do.

(c) (5 points) Consider $f(x) = x^2 - 2$ which has a root at $\sqrt{2}$. Take initial guess $x_0 = 1$ and $x_1 = 2$.

- Compute the first divided difference $f[x_0, x_1]$.
- Use the Secant method formula from part (b) to compute the next iterate x_2.

$$f[x_0,x_1] = \frac{f(x_1)-f(x_0)}{x_1-x_0} = \frac{2+1}{1} = 3$$

og secant

$$x_2 = x_1 - \frac{f(x_1)(x_1-x_0)}{f(x_1)-f(x_0)} = 2 - \frac{2(1)}{2-(-1)} = \frac{4}{3}$$

our secant

$$x_2 = x_0 - \frac{f(x_0)(x_1-x_0)}{f(x_1)-f(x_0)} = 1 - \frac{(-1)}{3} = \frac{4}{3}$$

(d) (10 points) Let x be a *simple root* of f, i.e., $f(x) = 0$, and $f'(x) \neq 0$. Suppose $\{x_n\}$ is generated by the secant method. Define the errors $e_n = x - x_n$.

- Starting from the secant iteration from part (b) and the definition of e_n, rewrite e_{n+1} in a form that is a product of the errors e_n, e_{n-1} and appropriate Newton divided differences of f.
- Using the exact characterizations of the divided differences from part (a), replace these divided differences by exact expressions for each.
- Using this error relation, give a brief qualitative comparison of the rate of convergence of the Secant method with the bisection method and Newton's method. (You do not need to compute an exact order r; just explain whether Secant appears to converge slower/faster than each, and why.)

$$e_{n-1} = x - x_{n-1}$$

$$e_n = x - x_n = x - x_{n-1} + f(x_{n-1})\frac{x_n - x_{n-1}}{f(x_n - f(x_{n-1})} = e_{n-1} + \cancel{f(x)}x_n$$

$$e_{n+1} = x - x_{n+1}$$

$$x_{n+1} = x_{n-1} - f(x_{n-1})\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

$$f[x_0,x_1] = \frac{f(x_1)-f(x_0)}{x_1-x_0}$$

Bisection method converges linearly it's very slow but it's graranteed. Newton's meth in comparison converges quadraticly ~~it~~ it costs more. & can fail if $f'(r) \neq 0$ Secant is super lineal, middle of Newton's & Bisecat

Bisection $\leq$ secant $\leq$ Newton

in terms of speed of convergence as newton needs f'

2. (25 points) **Numerical Differentiation:** The goal of this question is to derive and use finite-difference formulas for approximating derivatives.

   (a) (5 points) Let f be sufficiently smooth and let the three nodes be $x_0 - h, x_0, x_0 + h$ for $h > 0$.

   - Construct the quadratic interpolating polynomial $p(x)$ that interpolates f at these three points. You may use any form you like (Newton, Lagrange, or an equivalent form).
   - Differentiate $p(x)$ and evaluate at x_0 to obtain a finite-difference approximation for $f'(x_0)$ in the form

$$f'(x_0) \approx \alpha f(x_0 - h) + \beta f(x_0) + \gamma f(x_0 + h),$$

   and determine the coefficients α, β, γ.

$$f(x) \approx P_n(x) = f(x_0-h)\frac{(x-x_0)(x-(x_0+h))}{(x_0-h-x_0)(x_0-h-x_0-h)} + f(x_0)\frac{(x-(x_0-h))(x-(x_0+h))}{(x_0-(x_0-h))(x_0-(x_0+h))} + f(x_0+h)\frac{(x-x_0)(x-(x_0-h))}{(x_0+h-x_0)(x_0+h-(x_0-h))}$$

$$= f(x_0-h)\frac{(x^2-2xx_0-xh+x_0^2+x_0h)}{2h^2} + f(x_0)\frac{(x^2-2x_0x \ldots)}{h^2} + f(x_0+h)\frac{(x^2-2x_0x+xh+c)}{2h^2}$$

$$f'(x) \approx P_n'(x) = f(x_0-h)\frac{(2x-2x_0-h)}{2h^2} + f(x_0)\frac{(2x-2x_0)}{h^2} + f(x_0+h)\frac{(2x-2x_0+h)}{2h^2}$$

$$f'(x_0) = f(x_0-h)\left(-\frac{1}{2h}\right) + f(x_0+h)\left(\frac{1}{2h}\right) = \frac{f(x_0+h)-f(x_0-h)}{2h}$$

   (b) (7 points) Derive an expression for the difference $f'(x_0) - p'(x_0)$. From your expression, state clearly which derivative of f controls the error, and the order of accuracy of your formula with respect to h.

$$f(x) - P_2(x) = \frac{f^{(3)}(\xi)}{3!}\psi_3$$

$$f'(x) - P_2'(x) = \frac{d}{dx}\frac{f^{(3)}(\xi)}{6}\psi_3$$

$$= \underbrace{\psi_3'(x)\frac{f^{(3)}(\xi)}{6}}_{\text{dominant \& controls the error } \cdot\, O(h^2)} + \underbrace{\psi_3(x)\frac{f^{(4)}(\xi)}{24}}_{O(h^3)\ \text{\& only accounted for when } \psi_3'=0}$$

$\therefore$ symmetry makes $\psi_3'(x_0) = 0$ (as it does in centered schemes) & this scheme & leading term disappears. So the acc is $O(h^3)$

(c) (8 points) Let $f(x) = e^x$ and $x_0 = 0$, so that $f'(0) = 1$.

- Use your formula from part (a) with $h = 0.1$ to approximate $f'(0)$.
- Repeat this for $h = 0.05$
- Compute the absolute errors in both cases.
- Compare the two errors and briefly comment on whether the observed behavior is consistent with the order of accuracy you found in part (b).

You may leave your answers and errors in terms of exponentials (e.g. $e^0.1$, and $e^{0.05}$); no decimal arithmetic is required. You do not need to simplify the errors numerically.

$$P_2'(0) = f(-0.1)\frac{1}{-0.2} + f(0)(0) + f(0.1)\frac{1}{0.2} = -5f(-0.1) + 0 + 5f(0.1)$$
$$= -5e^{-0.1} + 5e^{0.1}$$

$$P_2'(0) = f(-0.05)\frac{1}{-0.1} + f(0)(0) + f(0.05)\frac{1}{0.1} = -10e^{-0.05} + 10e^{0.05}$$

$$\text{Abs Error}_1 = |1 - (5(e^{0.1} - e^{-0.1}))|$$

$$\text{Abs Error}_2 = |1 - (10(e^{0.05} - e^{-0.05}))|$$

we can see that Abs Error_2 is way smaller than Error_1 & this trakes with our $O(h^3)$ error from part b

$h_1^3 = 0.001$

$h_2^3 = 0.000125$

(d) (5 points) In exact arithmetic, making h small always reduces the truncation error in finite-difference formulas. In practice, computations are done in finite precision.

- Give a short explanation why taking h too small can actually make a numerical derivative less accurate on a computer.
- Explain how this interacts with the order of accuracy you found in part (b).

On a computer numbers are stored in bits & there's only a certain amt they store before the number is truncated & cut off. When h is really small when $h \to 0$ the ~~numbers~~ FLOPs also become very small so the numbers get truncated & this can cause error increase & interfer with the acc in b)

3. (25 points) **Extrapolation and Romberg Integration:** The goal of this question is to connect extrapolation and Romberg integration to *classical tableau methods* that we observed in Hermite interpolation.

   (a) (8 points) Consider the integral
   $$I = \int_0^1 e^x \, dx.$$
   Let $T(h)$ denote the composite trapezoidal rule approximation to I on $[0, 1]$ with step size h.
   1. Compute $T(1)$ using a single subinterval $[0, 1]$.
   2. Compute $T(\frac{1}{2})$ using two equal subintervals $[0, \frac{1}{2}]$ and $[\frac{1}{2}, 1]$.
   3. Leave your answers in exact form (you may use e and $e^{1/2}$; no decimal arithmetic is required).

1. $T(1) = \int_0^1 e^x dx = \frac{h}{2}[f(0) + f(1)] = \frac{1}{2}[1 + e^1]$

2. $T(\frac{1}{2}) = \frac{h}{2}[f(0) + 2f(\frac{1}{2}) + f(1)] = \frac{1}{4}[1 + 2e^{0.5} + e]$

   (b) (9 points) For sufficiently smooth f, the error of the composite trapezoidal rule on a fixed interval can be written in the form
   $$I - T(h) = Ch^2 + Dh^4 + \mathcal{O}(h^6),$$
   for some constants C and D independent of h.
   We now look for a new approximation $R(h)$ built from $T(h)$ and $T(h/2)$ whose error is of order h^4.
   1. Using the error expansion above, write down the corresponding expansion for $I - T(\frac{h}{2})$ in powers of h.
   2. Consider a linear combination
   $$R(h) = AT(\tfrac{h}{2}) + BT(h),$$
   where A and B are constants. Use your two error expansions to determine A and B so that the h^2 term cancels in $I - R(h)$.
   3. With your choices of A and B, show that the leading error term in $I - R(h)$ is of order h^4. (You may leave the coefficient of h^4 in terms of C and D.)

1. $I - T(\frac{h}{2}) = C(\frac{h}{2})^2 + D(\frac{h}{2})^4 + O(h^6) = \frac{Ch^2}{4} + \frac{Dh^4}{16} + O(h^6)$

2. $4I - 4T(\frac{h}{2}) = Ch^2 + \frac{Dh^4}{4} + O(h^6)$

$4I - I = 4T(\frac{h}{2}) - T(h) + Ch^2 - Ch^2 + \frac{Dh^4}{4} - Dh^4 + O(h^6)$

$I = \frac{4}{3}T(\frac{h}{2}) - \frac{T(h)}{3} - \frac{Dh^4}{4} + O(h^6)$

$A = \frac{4}{3}, B = -\frac{1}{3}$

$\therefore$ Now can be seen the leading error term is $O(h^4)$

(c) (4 points) For the integral in part (a), take $h = 1$ and use your formula for $R(h)$ from part (b) to compute $R(1)$ in terms of $T(1)$ and $T(\frac{1}{2})$.

1. Compute $R(1)$ explicitly using your values of $T(1)$ and $T(\frac{1}{2})$ from part (a). (Again, you may leave your answer in terms of e and $e^{1/2}$.)
2. The exact value of the integral is $I = e - 1$. Compare $R(1)$ with $T(1)$ and $T(\frac{1}{2})$ in terms of which is closer to $e - 1$, and briefly explain how this relates to the order of accuracy you found in part (b).

1. $R(1) = \frac{4}{3}T(\frac{h}{2}) - \frac{1}{3}T(h) = \frac{1}{3}[1 + 2e^{0.5} + e] - \frac{1}{6}[1 + e] = \frac{1}{6} + \frac{4}{6}e^{0.5} + \frac{e}{6}$

2. $T(1) = \frac{1}{2}[1 + e]$

$T(\frac{1}{2}] = \frac{1}{4}(1 + 2e^{0.5} + e]$

$R[1] = \frac{1}{6}[1 + 4e^{0.5} + e]$

$h^4 = 1$, the absolute error for $R[1]$ ~~should~~ would ~~since all values are theoretic~~ be the best. since $h=1$ ~~even though~~ all the $h^2, h^4 = 1$ & the accuracy isnt that good.

(d) (4 points) Recall that in Hermite interpolation we organized function values and derivative information in a tableau (divided-difference table) to systematically build higher-order approximations without recomputing everything from scratch. Romberg integration uses a similar tableau, built from trapezoidal approximations.

For the integral in part (a):

1. Sketch the first two columns of a Romberg tableau whose first column consists of $T(1)$, $T(\frac{1}{2})$, and $T(\frac{1}{4})$. Indicate where the value $R(1)$ from part (c) would appear in this tableau (specify its position in terms of row/column).
2. In 2–3 sentences, describe how the Romberg tableau plays a role similar to the Hermite (divided-difference) tableau: explain how each move to the right in the Romberg table increases the formal order of accuracy while reusing information from the previous column.

$R_{1,1} = T(1)$

row 2, col 2 $\to$ (c)

$R_{2,1} = T(\frac{1}{2}) \qquad R_{2,2} = R_{2,1} + \frac{1}{3}(R_{2,1} - R_{1,1}) = T(\frac{1}{2}) + \frac{1}{3}(T(\frac{1}{2}) - T(1)) = R(1)$

$R_{3,1} = T(\frac{1}{4}) = \frac{1}{8}[1 + 2e^{0.25} + 2e^{0.5} + 2e^{0.75} + e] \qquad R_{3,3} = R_{3,2} + \frac{1}{15}(R_{3,2} - R_{2,2})$

$R_{3,2} = R_{3,1} + \frac{1}{3}(R_{3,1} - R_{2,1})$

col 1	col 2	col 3
R_{11}		
R_{21}	R_{22}	
R_{31}	R_{32}	R_{33}

- Romberg integration uses both composite trapezoid rule & richardsons extrapolation for a very useful method. For composite trapezoid rule the ~~error~~ error expansion only includes even powers of h so this is esp effective as then each extrapolation step (richardsons) cancels the next leeding term in the error expansion, producing ~~a~~ succesively higher order approximations like Hermite (table). Each col has h^2 greater accuracy than the one to it's left. ~~a~~ col 1 $O(h^2)$ col 2 $O(h^4)$ col 3 $O(h^6)$

4. (25 points) **Solving Linear Systems with Special Structure** The goal of this problem is to analyze special structured matrices for solving the linear system $\mathbf{Ax} = \mathbf{b}$.

   (a) (5 points) Determine the conditions for a *general matrix* $\mathbf{A}$ to satisfy the following decompositions:

   - State a standard condition on $\mathbf{A}$ (in terms of its leading principal submatrices $A_{1:k,1:k}$) that guarantees an LU decomposition exists and is unique.
   - State a necessary and sufficient condition on $\mathbf{A}$ for the Cholesky factorization $\mathbf{A} = \mathbf{LL}^T$ to exist and be unique.

   In your answer, indicate the general form of $\mathbf{L}$ and $\mathbf{U}$.

(ii) For Cholesky $A = LL^T$ factorization matrix A must be positive definite aka it needs to be symmetric & $x^T A x > 0$ for every n dimensional vector $x \neq 0$. $A = LL^T$ where, then L is a lower triangular with non zero diagonal entries.

   (b) (5 points) Consider the specific system

$$\mathbf{Ax} = \mathbf{b}, \quad \mathbf{A} = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

   1. Show that $\mathbf{A}$ satisfies the condition you stated in part (a) for an LU decomposition (without pivoting) to exist.
   2. Show that $\mathbf{A}$ satisfies the condition for a Cholesky decomposition by writing $\mathbf{x}^\top \mathbf{Ax}$ as a *sum of squares*.

1.

2. $$Ax = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2a-b \\ -a+2b-c \\ -b+2c-d \\ -c+2d \end{bmatrix}$$

$$x^T A x = [a \; b \; c \; d] \begin{bmatrix} 2a-b \\ -a+2b-c \\ -b+2c-d \\ -c+2d \end{bmatrix} = 2a^2 - ab - ab + 2b^2 - bc - cb + 2c^2 - cd - cd + 2d^2$$

$$= 2a^2 - 2ab + 2b^2 - 2bc + 2c^2 - 2cd + 2d^2$$

$$= 2[$$

$-1 + \frac{3}{2} \cdot \frac{2}{3} = 0$

(c) (7 points) Derive the exact form of **L** and **U** for the matrix **A** provided in part (b). You may use the algorithm of your preference.

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix}$$

$R_2 - (-\frac{1}{2})R_1$ so $l_{21} = -\frac{1}{2}$, $u_{22} = \frac{3}{2}$

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 0 & 3/2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

$R_3 - (-\frac{2}{3})R_2$ so $l_{32} = -\frac{2}{3}$

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 0 & 3/2 & -1 & 0 \\ 0 & 0 & 4/3 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

$R_4 - (-\frac{3}{4})R_3$ so $l_{43} = -\frac{3}{4}$

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 0 & 3/2 & -1 & 0 \\ 0 & 0 & 4/3 & -1 \\ 0 & 0 & 0 & 5/4 \end{bmatrix}$$

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1 & 0 & 0 \\ 0 & -2/3 & 1 & 0 \\ 0 & 0 & -3/4 & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 0 & 3/2 & -1 & 0 \\ 0 & 0 & 4/3 & -1 \\ 0 & 0 & 0 & 5/4 \end{bmatrix}$$

(d) (8 points) In this part, you will derive a Cholesky factorization of **A** starting from your LU factors, and use it to solve $\mathbf{Ax} = \mathbf{b}$.

1. Using your matrices **L** and **U** from part (c), find a diagonal matrix **D** with positive diagonal entries such that $\mathbf{U} = \mathbf{D}\mathbf{L}^\top$. (Hint: equate the entries of **U** and $\mathbf{DL}^\top$ and solve for the diagonal entries of **D**.)
2. Use your **D** to write **A** in the form $\mathbf{A} = \mathbf{LDL}^\top$. Define a new lower triangular matrix $\tilde{\mathbf{L}} = \mathbf{LD}^{1/2}$, where $\mathbf{D}^{1/2}$ is the diagonal matrix whose entries are the square roots of the diagonal entries of **D**. Show that $\mathbf{A} = \tilde{\mathbf{L}}\tilde{\mathbf{L}}^\top$. (This is a Cholesky factorization of **A**.)

$$U = DL^T = \begin{bmatrix} d_1 & 0 & 0 & 0 \\ 0 & d_2 & 0 & 0 \\ 0 & 0 & d_3 & 0 \\ 0 & 0 & 0 & d_4 \end{bmatrix} \begin{bmatrix} 1 & -1/2 & 0 & 0 \\ 0 & 1 & -2/3 & 0 \\ 0 & 0 & 1 & -3/4 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{cases} d_1 = 2 \\ -\frac{1}{2}d_1 = -1 \\ d_2 = 3/2 \\ d_3 = 4/3 \end{cases}$$

$d_4 = 5/4$

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3/2 & 0 & 0 \\ 0 & 0 & 4/3 & 0 \\ 0 & 0 & 0 & 5/4 \end{bmatrix}$$

2. $$A = LU = LDL^T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1 & 0 & 0 \\ 0 & -2/3 & 1 & 0 \\ 0 & 0 & -3/4 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3/2 & 0 & 0 \\ 0 & 0 & 4/3 & 0 \\ 0 & 0 & 0 & 5/4 \end{bmatrix} \begin{bmatrix} 1 & -1/2 & 0 & 0 \\ 0 & 1 & -2/3 & 0 \\ 0 & 0 & 1 & -3/4 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\tilde{L} = LD^{1/2} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/2 & 1 & 0 & 0 \\ 0 & -2/3 & 1 & 0 \\ 0 & 0 & -3/4 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{2} & 0 & 0 & 0 \\ 0 & \sqrt{3/2} & 0 & 0 \\ 0 & 0 & \sqrt{4/3} & 0 \\ 0 & 0 & 0 & \sqrt{5/4} \end{bmatrix} = \begin{bmatrix} \sqrt{2} & 0 & 0 & 0 \\ -\frac{\sqrt{2}}{2} & \sqrt{\frac{3}{2}} & 0 & 0 \\ 0 & -\frac{2}{3}\sqrt{\frac{3}{2}} & \sqrt{\frac{4}{3}} & 0 \\ 0 & 0 & -\frac{3}{4}\sqrt{\frac{4}{3}} & \sqrt{\frac{5}{4}} \end{bmatrix}$$

$$\tilde{L}\tilde{L}^T = \begin{bmatrix} \sqrt{2} & 0 & 0 & 0 \\ -\frac{\sqrt{2}}{2} & \sqrt{\frac{3}{2}} & 0 & 0 \\ 0 & -\frac{2}{3}\sqrt{\frac{3}{2}} & \sqrt{\frac{4}{3}} & 0 \\ 0 & 0 & -\frac{3}{4}\sqrt{\frac{4}{3}} & \sqrt{\frac{5}{4}} \end{bmatrix} \begin{bmatrix} \sqrt{2} & -\frac{\sqrt{2}}{2} & 0 & 0 \\ 0 & \sqrt{3/2} & -\frac{2}{3}\sqrt{\frac{3}{2}} & 0 \\ 0 & 0 & \sqrt{4/3} & -\frac{3}{4}\sqrt{\frac{4}{3}} \\ 0 & 0 & 0 & \sqrt{\frac{5}{4}} \end{bmatrix} = \begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} = A$$

$\therefore A = \tilde{L}\tilde{L}^T$

Feel free to use this as scratch.

Absolute error $e = |x - \hat{x}|$

Relative error: $rel(x) = \frac{|x-\hat{x}|}{|x|}$

Forward error: explains how far the computed output is from the true output

Backward error: find the smallest change to the input that explains computed output

$(A + \Delta A)\hat{x} = b$

Condition # $\kappa(f,x) = \left|\frac{x f'(x)}{f(x)}\right|$

for matrix: $\kappa(A) = \|A\|\|A^{-1}\|$

- ↑κ# means: small changes in input → big in output

eg Vandermonde problem is ill conditioned, not algo

Fixed point: $x_{n+1} = g(x_n)$

existance: If $g \in C[a,b]$ & $g(x) \in [a,b]\ \forall x \in [a,b]$ then g has at least 1 fixed point in $[a,b]$

uniqueness: If, in addition, $g'(x)$ exists & ∃ constant $k<1$ s.t. $|g'(x)| \le k\ \forall x \in (a,b)$ then only 1 fp.

converges if: $|g'(r)| < 1$: $g'(r) = 0$ & $g'(r) \neq 0$ fuq, quadratic

Error: $|p_n - p| \le \frac{k}{1-k}|p_1 - p_0|$ or $e_{n+1} \approx g'(r)e_n$

Root Finding:

- Bisection: $f \in C[a,b]$ s.t. $f(a)\cdot f(b) < 0$
  - gaurateed convergence but slow
  - Error: $|x_n - r| \le \frac{b-a}{2^n}$
- Newtons Method: $g(x) = x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$
  - quadratic convergence if $f'(r) \neq 0$
  - need to start close to root
  - fail if: derivative zero, tangent nearly horizontal
  - converges if: $|p - p_0| < \frac{2\min|f'(x)| > 0}{\max|f''(x)| < \infty}$
- Secant Method: $g(x) = x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$
  - superlinear (≈ 1.618), no derivative needed
  - \+ faster than fixed point, cheaper than Newton
  - Not guaranteed, needs 2 starting values

Interpolation

- Vandermonde Formation: $(x_0, y_0) \cdots (x_n, y_n)$

interpolation poly: $p(x) = a_0 + a_1 x + \cdots + a_n x^n$, $p(x_i) = y_i$

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & & & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{bmatrix}\begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix}$$

$V_{i,j} = x_i^j$ for $i,j = 0,\dots,n$

- $\det(V) = \prod_{0 \le i < j \le n}(x_j - x_i)$ if 0 interp impossible (equal notes)
- nodes badly spaced or n↑ → unstable & runge phenomenum
- large powers → round off errors & numerically disasterous
- small diff → instability.

Thus

$f'(x) - p_n'(x) =$

→ Hermite: Newton's with double points

$H(x_i) = f(x_i)$ & $H'(x_i) = f'(x_i)$; $f[x_0, x_0] = f'(x_0)$

- $2(n+1)$ constraints ⇒ degree of polynomial $\le 2n+1$

Error: $\frac{f^{(2n+2)}(\xi)}{(2n+2)!}\prod_i (x - x_i)^2$. matches f & f' vals so reduces error

→ Splines: order $m \ge 1$, subintervals $[x_{i-1}, x_i]$

(i) $s(x)$ is a polynomial of degree $< m$ on ∀ interval

(ii) $\forall r, 0 \le r \le m-2$ $s^{(r)}(x) \in C[a,b]$ or $S \in C^{m-2}[a,b]$

- use low degree poly locally, enforce smoothness across int
- maintain good approx, avoid runge, accurate &

\+ $S(x_j) = f(x_j)\ \forall j = 0,\dots,n$; $S_j^{(i)}(x_{j+1}) = S_{j+1}^{(i)}(x_{j+1})\ \forall j = 0,\dots,n-2$

- natural: $S''(x_0) = S''(x_n) = 0$ & clamped: $S'(x_0) = f'(x_0)$, $S'(x_n) = f'(x_n)$
- smoothest possible func that satisfies interpol
- $S_j(x) = f(x_j) + b_j(x - x_j) + \cdots + d_j(x - x_j)^3$
- Error cubic: $f(x) - S(x) = \frac{f^{(4)}(\xi)}{384}(x - x_i)^2(x_{i+1} - x)^2 \approx \underline{O(h^4)}$

* Differentiation

- forward: $f'(x) \approx \frac{f(x+h) - f(x)}{h}$ $O(h)$. As $h \to 0$ truncation error↓
- backward: $f'(x) \approx \frac{f(x) - f(x-h)}{h}$ round off error ↑
- central: $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$ $O(h^2)$ (only even)
- 3point endpoint: $f'(x_0) \approx \frac{-3f_0 + 4f_1 - f_2}{2h}$ $O(h^2)$
- 3 point centered: $f'(x_i) \approx \frac{f_{i+1} - f_{i-1}}{2h}$ $O(h^2)$
- centered have symmetric error cancellation
- endpoints introduce bias
- constant multiplying with h^2 smaller

* Richardson Extrapolation

- $A(h) = T + Ch^p + O(h^{p+1})$ do $A(\frac{h}{2})$

$R/T = \frac{2^p A(h/2) - A(h)}{2^p - 1} + O(h^{p+1})$ (for centred $p=2$, h^4)

→ Interpolation of Derivatives

$f'(x) \approx p_n'(x) = \sum_{j=0}^{n} f(x_j)\ell_j'(x)$

Error: $f'(x) - P_n'(x) = \frac{d}{dx}\frac{f^{(n+1)}(\xi)}{(n+1)!}\Psi_n(x)$

dominant → $= \Psi_n'(x)\frac{f^{(n+1)}}{(n+1)!}(\xi) + \Psi_n(x)\frac{f^{(n+2)}}{(n+2)!}(\xi)$

$O(h^n)$ $O(h^{n+1})$

→ Lagrange Polynomial: $L_i(x) = \prod_{j=0, j\neq i}^{n}\frac{x - x_j}{x_i - x_j} = \frac{\Psi_n(x)}{(x - x_i)\Psi_n'(x_i)}$

$P_n(x) = \sum_{i=0}^{n} y_i L_i(x)$

has degree $\le n$ (n+1 points)

— bad spacing → Runge

- Error $\boxed{f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\Psi_n}$

→ Divided/Newtons: basis $1, (x - x_0), (x - x_0)(x - x_1), \dots$

$P_n(x) = a_0 + a_1(x - x_0) + \cdots + a_n(x - x_0)\cdots(x - x_{n-1})$

$a_n = f[x_0, \dots, x_n] = \frac{f[x_1, \dots, x_n] - f[x_0, x_1, \dots, x_{n-1}]}{x_n - x_0}$

$\boxed{P_n(x) = f(x_0) + \sum_{k=1}^{n} f[x_0 \cdots x_k](x - x_0)\cdots(x - x_{k-1})}$

For numerical differentiation, a 3-point scheme means we approximate $f'(x_0)$ using the function values at three nodes (sample points) that are evenly spaced by h. The classic endpoint (one-sided) schemes are the forward and backward differences: the 3-point forward formula is $f'(x_0) \approx \frac{-3f(x_0) + 4f(x_0+h) - f(x_0+2h)}{2h}$ and the 3-point backward formula is $f'(x_0) \approx \frac{3f(x_0) - 4f(x_0-h) + f(x_0-2h)}{2h}$; using Taylor series about x_0, you can show both have error $\sim C\,h^2$, so they are second-order accurate. The centered 3-point scheme uses one point on each side, $x_0 - h, x_0, x_0 + h$, and gives $f'(x_0) \approx \frac{f(x_0+h) - f(x_0-h)}{2h}$; expanding $f(x_0 \pm h) = f(x_0) \pm hf'(x_0) + \frac{h^2}{2}f''(x_0) \pm \frac{h^3}{6}f^{(3)}(x_0) + \cdots$, the even powers cancel in the numerator and we get $f'(x_0) + O(h^2)$, so this is also second-order, but with a smaller leading error constant and, more generally, higher symmetry. In the interpolation picture, we build a polynomial $p_n(x)$ through the $n+1$ nodes and use $p_n'(x_0)$ as our derivative approximation; the error has the form $f'(x_0) - p_n'(x_0) = \Psi_n'(x_0)\frac{f^{(n+1)}(\xi_1)}{(n+1)!} + \Psi_n(x_0)\frac{f^{(n+2)}(\xi_2)}{(n+2)!}$, where $\Psi_n(x) = \prod_{j=0}^{n}(x - x_j)$ is the nodal product. For evenly spaced nodes $x_j = x_0 + jh$, each $(x_0 - x_j)$ is roughly a multiple of h, so $\Psi_n(x_0) \sim h^{n+1}$ and $\Psi_n'(x_0) \sim h^n$, the leading term in the error is controlled by $\Psi_n'(x_0)$, and if symmetry makes $\Psi_n'(x_0) = 0$ (as in centered schemes), that entire leading error term disappears and the error effectively drops by one power of h. Intuitively, endpoint schemes "look" only to one side, so the first-order asymmetry term survives in the error expansion, whereas centered schemes "look" both left and right, causing low-order error terms to cancel and making them more accurate and more stable in practice for the same step size h.

Setup. Let f be continuous on $[a,b]$ with $f(a)f(b) < 0$. (Assume a unique root $p \in (a,b)$.)
Iteration. Initialize $a_1 = a$, $b_1 = b$. For $n \ge 1$:

$$p_n = a_n + \frac{b_n - a_n}{2} = \frac{a_n + b_n}{2}.$$

(note the first is expression more numerically stable).

$$\begin{cases} \text{if } f(p_n) = 0 \text{ then stop (root found);} \\ \text{if } f(a_n)f(p_n) < 0 \Rightarrow a_{n+1} = a_n,\ b_{n+1} = p_n; \\ \text{else } a_{n+1} = p_n;\ b_{n+1} = b_n. \end{cases}$$

Analyzing the Bisection Algorithm

Theorem 1. *Suppose $f \in C[a,b]$ and $f(a)f(b) < 0$. The Bisection method generates a sequence $\{p_n\}_{n=1}^{\infty}$ ap- mating a zero p of f with*

$$|p_n - p| \le \frac{b-a}{2^n}, \qquad n \ge 1.$$

Definition 1 (Fixed point). *A number p is a* **fixed point** *of a function g if $g(p) = p$.*

Equivalence with root-finding. Root-finding $f(p) = 0$ and fixed-point problems $g(p) = p$ are changeable:

- Given $f(p) = 0$, define g so that p is a fixed point, e.g.

$$g(x) = x - f(x) \qquad \text{or} \qquad g(x) = x + 3f(x).$$

  Then $g(p) = p$ whenever $f(p) = 0$.

- Conversely, if g has a fixed point p ($g(p) = p$), then

$$f(x) = x - g(x)$$

  satisfies $f(p) = 0$.

Theorem 2 (Fixed Point Existence and Uniqueness). *(i) If $g \in C[a,b]$ and $g(x) \in [a,b]$ for all $x \in [a,b]$, g has at least one fixed point in $[a,b]$.*
(ii) If, in addition, $g'(x)$ exists on (a,b) and there is a constant $k < 1$ such that

$$|g'(x)| \le k \qquad \text{for all } x \in (a,b),$$

then there is exactly one fixed point in $[a,b]$.

Fixed Point Approximation

To approximate a fixed point of a function g, choose an initial guess p_0 and generate the sequence $\{p_n\}$ by

$$p_n = g(p_{n-1}), \qquad n \ge 1.$$

If the sequence converges to p and g is continuous, then

$$p = \lim_{n\to\infty} p_n = \lim_{n\to\infty} g(p_{n-1}) = g\left(\lim_{n\to\infty} p_{n-1}\right) = g(p),$$

so p solves $x = g(x)$. This procedure is called *fixed-point* (or *functional*) iteration.

[H] Fixed-Point (Functional) Iteration for $p = g(p)$
Require: function g, initial guess p_0, tolerance TOL, max iterations N_{max}
Ensure: approximate fixed point p or failure

```
i ← 1
while i ≤ N_max do
    p ← g(p_0)                         ▷ comput
    if |p − p_0| < TOL then
        return p                       ▷ succ
    end if
    p_0 ← p                            ▷ update ite
    i ← i + 1
end while
return failure: no convergence within N_max iterations
```

- flexible
- can be adapted
- doesnt always work
- linear convergence ($g'(x) \neq 0$)

Newton's Method and Its Extensions

Assume $f \in C^2[a,b]$. Let $p_0 \in [a,b]$ be an approximation to a root p with $f'(p_0) \neq 0$ and $|p - p_0|$ small. first-order Taylor expansion of f about p_0, evaluated at $x = p$, gives

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2}f''(\xi(p)),$$

where $\xi(p)$ lies between p and p_0. Since $f(p) = 0$,

$$0 = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2}f''(\xi(p)).$$

If $|p - p_0|$ is small, the quadratic term is much smaller, so

$$0 \approx f(p_0) + (p - p_0)f'(p_0).$$

Theorem 4 (Local convergence of Newton's method). *Let $f \in C^2[a,b]$. If $p \in (a,b)$ satisfies $f(p) = 0$, $f'(p) \neq 0$, then there exists $\delta > 0$ such that Newton's method generates a sequence $\{p_n\}_{n=1}^{\infty}$ converging to p for initial approximation $p_0 \in [p - \delta, p + \delta]$.*

Theorem 5 (Newton step and local error). *Assume $f \in C^2[a,b]$ and $p \in [a,b]$ is a simple root: $f(p)$, $f'(p) \neq 0$. For p_0 sufficiently close to p, define the Newton iterates*

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)}.$$

Then for each n there exists ξ_n between p_n and p such that

$$p - p_{n+1} = -\frac{f''(\xi_n)}{2f'(p_n)}(p - p_n)^2.$$

This follows directly from the Taylor series expansion.

- from fixed pointer
- new mutation when $g'(p) = 0$
- costly to compute $f'(x)$
- only local convergence

Theorem 6 (Quadratic convergence of Newton's method). *Assume $f \in C^2[a,b]$ and let $p \in (a,b)$ so $f(p) = 0$ and $f'(p) \neq 0$. If p_0 is sufficiently close to p, the Newton iterates*

$$p_{n+1} = p_n - \frac{f(p_n)}{f'(p_n)}, \qquad n \ge 0,$$

are well defined, satisfy $p_n \to p$, and

$$\lim_{n\to\infty}\frac{p - p_{n+1}}{(p - p_n)^2} = -\frac{f''(p)}{2f'(p)}.$$

In particular, the order of convergence is $p = 2$.

Suppose that $f \in C^n[a,b]$ and $x_0, x_1, \dots, x_n$ are distinct numbers in $[a,b]$. Then a number ξ exists in (a,b) with

$$f[x_0, x_1, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}.$$

Newton's method is powerful but requires evaluating f' at each step. To avoid derivative evaluations, approximate f' by a secant slope. By definition,

$$f'(p_{n-1}) = \lim_{x \to p_{n-1}} \frac{f(x) - f(p_{n-1})}{x - p_{n-1}}.$$

from Newton
+ no derivative needed
- slower

If p_{n-2} is close to p_{n-1}, then

$$f'(p_{n-1}) \approx \frac{f(p_{n-1}) - f(p_{n-2})}{p_{n-1} - p_{n-2}}.$$

Definition. Substituting this in Newton's update yields the *secant* iteration

$$p_n = p_{n-1} - \frac{f(p_{n-1})(p_{n-1} - p_{n-2})}{f(p_{n-1}) - f(p_{n-2})}, \qquad n \geq 2. \tag{2}$$

Geometric interpretation. Starting from two initial approximations p_0 and p_1:

- p_2 is the x-intercept of the line through $(p_0, f(p_0))$ and $(p_1, f(p_1))$.
- p_3 is the x-intercept of the line through $(p_1, f(p_1))$ and $(p_2, f(p_2))$, etc.

The Method of False Position

Idea. Like the Secant method, use the secant line to estimate the root, but *always keep the root bracketed*. Start with a, b such that $f(a)f(b) < 0$. At each step form the secant through $(a, f(a))$ and $(b, f(b))$ and take its x-intercept as the next approximation. Replace the endpoint that has the *same* sign as the new value so that the sign change (and hence the bracket) is preserved.

Bisection + Secant
- Use secant line
Add bracketing constraint
guaranteed con

Secant intercept. Given current bracket $[a, b]$ with $f(a)f(b) < 0$, define

$$p = b - \frac{f(b)\,(b-a)}{f(b) - f(a)} = a - \frac{f(a)\,(a-b)}{f(a) - f(b)}.$$

Bracketing update.

$$\text{If } f(a)\,f(p) < 0 \text{ then } b \leftarrow p; \qquad \text{else } a \leftarrow p.$$

This guarantees $f(a)f(b) < 0$ after every iteration.

Algorithm 3 False Position

Require: function f, initial a, b with $f(a)f(b) < 0$, tolerance TOL, max its $N_{\max}$
Ensure: approximate root p (bracket maintained) or failure

```
for n = 1 to N_max do
    p ← b − f(b)(b − a)/(f(b) − f(a))          ▷ x-intercept of secan
    if |f(p)| < TOL or max(|p − a|, |b − p|) < TOL then
        return p
    end if
    if f(a) f(p) < 0 then
        b ← p
    else
        a ← p
    end if
end for
return failure: no convergence within N_max iterations
```

Remarks.

- Each step needs only one new function evaluation (after the first two).
- Bracketing is preserved, so a root remains in $[a, b]$; however, unlike bisection, the interval length need not halve each step.

Vandermonde formulation. Suppose we seek a polynomial

$$p(x) = a_0 + a_1 x + \cdots + a_m x^m.$$

Since (3.1.1) imposes $n + 1$ conditions on $p(x)$, it is reasonable to first consider the case $m = n$. Then we want to find $a_0, a_1, \ldots, a_n$ such that

$$a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0,$$
$$\vdots$$
$$a_0 + a_1 x_n + a_2 x_n^2 + \cdots + a_n x_n^n = y_n.$$

This is a system of $n+1$ linear equations in $n+1$ unknowns, completely equivalent to solving the polynomial interpolation problem. In vector–matrix notation, the system is

$$X a = y$$

with

$$X_{i,j} = [x_i^j], \qquad i, j = 0, 1, \ldots, n,$$

$$a = [a_0 \quad a_1 \quad \cdots \quad a_n]^T, \qquad y = [f(x_0) \quad \cdots \quad f(x_n)]^T.$$

The matrix X is called a *Vandermonde matrix*.

We say $s(x)$ is a *spline function of order* $m \geq 1$ if it satisfies:

(P1) On each subinterval $[x_{i-1}, x_i]$, $s(x)$ is a polynomial of degree $< m$.

(P2) For $0 < r < m - 2$, the derivative $s^{(r)}(x)$ is continuous on $[a, b]$ (i.e., $s \in C^{m-2}[a, b]$).

Oscillatory Behaviors. Let $f \in C^4[a, b]$ and let $x_0 = a < x_1 < \cdots < x_n = b$ be a partition. Let s_c be the *complete (clamped) cubic spline* interpolant of f satisfying

$$s_c(x_i) = f(x_i) \quad (i = 0, \ldots, n), \qquad s_c'(a) = f'(a), \quad s_c'(b) = f'(b).$$

Consider the admissible class

$$\mathcal{A} := \{\, g \in C^2[a, b] : g(x_i) = f(x_i) \ (i = 0, \ldots, n),\ g'(a) = f'(a),\ g'(b) = f'(b) \,\}.$$

Then

$$\int_a^b |s_c''(x)|^2\, dx \leq \int_a^b |g''(x)|^2\, dx, \qquad \forall g \in \mathcal{A}, \tag{10}$$

with equality iff $g \equiv s_c$.

x	$f(x)$	divided differences	divided differences	divided differences
x_0	$f[x_0]$			
		$f[x_0, x_1] = \frac{f[x_1] - f[x_0]}{x_1 - x_0}$		
x_1	$f[x_1]$		$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$	
		$f[x_1, x_2] = \frac{f[x_2] - f[x_1]}{x_2 - x_1}$		$f[x_0, x_1, x_2, x_3] = \frac{f[x_1, x_2, x_3] - f[x_0, x_1, x_2]}{x_3 - x_0}$
x_2	$f[x_2]$		$f[x_1, x_2, x_3] = \frac{f[x_2, x_3] - f[x_1, x_2]}{x_3 - x_1}$	
		$f[x_2, x_3] = \frac{f[x_3] - f[x_2]}{x_3 - x_2}$		$f[x_1, x_2, x_3, x_4] = \frac{f[x_2, x_3, x_4] - f[x_1, x_2, x_3]}{x_4 - x_1}$
x_3	$f[x_3]$		$f[x_2, x_3, x_4] = \frac{f[x_3, x_4] - f[x_2, x_3]}{x_4 - x_2}$	

$$P_n(x) = f[x_0] + \sum_{k=1} f[x_0, x_1, \ldots, x_k](x - x_0) \cdots (x - x_{k-1})$$

Example (Cubic Hermite interpolation). The most widely used form of Hermite interpolation is the cubic Hermite polynomial, which solves

$$p(a) = f(a), \qquad p'(a) = f'(a), \qquad p(b) = f(b), \qquad p'(b) = f'(b).$$

The divided–difference form becomes

$$H_2(x) = f(a) + (x-a)f'(a) + (x-a)^2 f[a, a, b] + (x-a)^2(x-b) f[a, a, b, b],$$

in which

$$f[a, a, b] = \frac{f[a, b] - f'(a)}{b - a}, \qquad f[a, a, b, b] = \frac{f'(b) - 2f[a, b] + f'(a)}{(b-a)^2}.$$

The error formula is

$$f(x) - H_2(x) = (x-a)^2(x-b)^2 f[a, a, b, b, x]$$
$$= \frac{(x-a)^2(x-b)^2}{24} f^{(4)}(\xi_x),$$

and hence

$$\max_{a \leq x \leq b} |f(x) - H_2(x)| \leq \frac{(b-a)^4}{384} \max_{a \leq t \leq b} |f^{(4)}(t)|.$$

$$H_{2n+1}(x) = f[z_0] + \sum f[z_0, \ldots, z_k](x - z_0)(x - z_1) \cdots (x - z_{k-1})$$

HERMITE

z	$f(z)$	differences	differences
$z_0 = x_0$	$f[z_0] = f(x_0)$		
		$f[z_0, z_1] = f'(x_0)$	
$z_1 = x_0$	$f[z_1] = f(x_0)$		$f[z_0, z_1, z_2] = \frac{f[z_1, z_2] - f[z_0, z_1]}{z_2 - z_0}$
		$f[z_1, z_2] = \frac{f[z_2] - f[z_1]}{z_2 - z_1}$	
$z_2 = x_1$	$f[z_2] = f(x_1)$		$f[z_1, z_2, z_3] = \frac{f[z_2, z_3] - f[z_1, z_2]}{z_3 - z_1}$
		$f[z_2, z_3] = f'(x_1)$	

RICHARDSONS EXTRAPOLATION

Suppose $x_0, x_1, \ldots, x_n$ are distinct numbers in the interval $[a, b]$ and $f \in C^{n+1}[a, b]$. Then, for each x in $[a, b]$, a number $\xi(x)$ (generally unknown) between $\min\{x_0, x_1, \ldots, x_n\}$, and the $\max\{x_0, x_1, \ldots, x_n\}$ and hence in (a, b), exists with

$$f(x) = P(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)(x - x_1) \cdots (x - x_n).$$

where $P(x)$ is the interpolating polynomial given in Eq. (3.1).

To see specifically how we can generate the extrapolation formulas, consider the $O(h)$ formula for approximating M

$$M = N_1(h) + K_1 h + K_2 h^2 + K_3 h^3 + \cdots. \tag{4.10}$$

The formula is assumed to hold for all positive h, so we replace the parameter h by half its value. Then we have a second $O(h)$ approximation formula

$$M = N_1\left(\frac{h}{2}\right) + K_1 \frac{h}{2} + K_2 \frac{h^2}{4} + K_3 \frac{h^3}{8} + \cdots. \tag{4.11}$$

Subtracting Eq. (4.10) from twice Eq. (4.11) eliminates the term involving K_1 and gives

$$M = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right] + K_2\left(\frac{h^2}{2} - h^2\right) + K_3\left(\frac{h^3}{4} - h^3\right) + \cdots. \tag{4.12}$$

Define

$$N_2(h) = N_1\left(\frac{h}{2}\right) + \left[N_1\left(\frac{h}{2}\right) - N_1(h)\right].$$

$$T = \frac{2^p (A(\tfrac{h}{2})) - A(h)}{2^p - 1} + O(h^{p+1})$$

Then Eq. (4.12) is an $O(h^2)$ approximation formula for M:

$$M = N_2(h) - \frac{K_2}{2} h^2 - \frac{3K_3}{4} h^3 - \cdots. \tag{4}$$

Richardson extrapolation is a trick for squeezing more accuracy out of a basic numerical method without redesigning it. Imagine you want some true value M (like an exact derivative or integral), but all you actually compute is an approximation $N(h)$ that depends on a step size h. We know this approximation is not perfect: its error can be written as $M - N(h) = K_1 h + K_2 h^2 + K_3 h^3 + \ldots$, so the main source of error is the first term $K_1 h$ (that's what "error is $O(h)$" means). The idea of Richardson extrapolation: if we can evaluate the same method with two different step sizes, say h and $h/2$, then we get two slightly different wrong answers whose errors we understand. By taking a clever linear combination of these approximations, we can make the $K_1 h$ terms cancel each other out, leaving a new approximation whose leading error now starts at h^2 instead of h. In other words, we build a new, more accurate estimate of M from the old low-order method just by doing some algebra on multiple runs with different h. This is why it's called "extrapolation": we use values at nonzero h to predict what the method would give in the ideal limit $h \to 0$, where the error disappears.